

**LAPORAN PRAKTIKUM  
STRUKTUR DATA**

**MODUL X  
PENGENALAN TREE**



**Disusun Oleh :**

NAMA : Rikza Nur Zaki

NIM : 103112430030

**Dosen**

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA  
FAKULTAS INFORMATIKA  
TELKOM UNIVERSITY PURWOKERTO  
2025**

#### A. Dasar Teori Tree

Tree adalah struktur data non-linear yang terdiri dari node dan hanya memiliki satu node akar (root) tanpa pendahulu. Tree tidak mengandung sirkuit. Binary Search Tree (BST) adalah jenis Tree di mana setiap node maksimal memiliki dua anak, dan aturannya adalah left child nilainya lebih kecil dari parent, sedangkan right child nilainya lebih besar. Operasi penting pada Tree melibatkan Insert, Search (menggunakan algoritma rekursif binary search), Delete, dan Traversal (seperti Pre-order, In-order, dan Post-order).

#### B. Guided (berisi screenshot source code & output program disertai penjelasannya)

##### Guided 1

```
tree.h
#ifndef TREE_H
#define TREE_H

struct Node {
    int data;
    Node *left, *right;
    int height;
};

class BinaryTree {
private:
    Node* root;

    Node* insertNode(Node* node, int value);
    Node* deleteNode(Node* node, int value);

    int getHeight(Node* node);
    int getBalance(Node* node);

    Node* rotateRight(Node* y);
    Node* rotateLeft(Node* x);

    Node* minValueNode(Node* node);

    void inorder(Node* node);
    void preorder(Node* node);
    void postorder(Node* node);

public:
    BinaryTree();
    void insert(int value);
    void deleteValue(int value);
    void update(int oldVal, int newVal);

    void inorder();
    void preorder();
    void postorder();
};

#endif

tree.cpp
```

```

#include "tree.h"
#include <iostream>
using namespace std;

BinaryTree::BinaryTree() {
    |   root = nullptr;
    }

int BinaryTree::getHeight(Node* n) {
    |   return (n == nullptr) ? 0 : n->height;
    }

int BinaryTree::getBalance(Node* n) {
    |   return (n == nullptr) ? 0 :
    |   getHeight(n->left) - getHeight(n->right);
    }

Node* BinaryTree::rotateRight(Node* y) {
    |   Node* x = y->left;
    |   Node* T2 = x->right;

    |   x->right = y;
    |   y->left = T2;

    |   y->height = max(getHeight(y->left),
    |   |   getHeight(y->right)) + 1;
    |   x->height = max(getHeight(x->left),
    |   |   getHeight(x->right)) + 1;

    |   return x;
    }

```

```

Node* BinaryTree::insertNode(Node* node, int value) {
    if (node == nullptr) {
        Node* newNode = new Node(value, nullptr, nullptr, 1);
        return newNode;
    }

    if (value < node->data)
        node->left = insertNode(node->left, value);
    else if (value > node->data)
        node->right = insertNode(node->right, value);
    else
        return node;

    node->height = 1 + max(getHeight(node->left),
        getHeight(node->right));

    int balance = getBalance(node);

    if (balance > 1 && value < node->left->data)
        return rotateRight(node);

    if (balance < -1 && value > node->right->data)
        return rotateLeft(node);

    if (balance > 1 && value > node->left->data) {
        node->left = rotateLeft(node->left);
        return rotateRight(node);
    }

    if (balance < -1 && value < node->right->data) {
        node->right = rotateRight(node->right);
        return rotateLeft(node);
    }

    return node;
}

void BinaryTree::insert(int value) {
    root = insertNode(root, value);
}

Node* BinaryTree::minValueNode(Node* node) {
    Node* current = node;
    while (current->left != nullptr)
        current = current->left;
    return current;
}

```

```

Node* BinaryTree::deleteNode(Node* root, int key) {
    if (root == nullptr)
        return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        if ((root->left == nullptr) || (root->right == nullptr)) {
            Node* temp = root->left ? root->left : root->right;

            if (temp == nullptr) {
                temp = root;
                root = nullptr;
            } else {
                *root = *temp;
            }
            delete temp;
        } else {
            Node* temp = minValueNode(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data);
        }
    }

    if (root == nullptr)
        return root;

    root->height = 1 + max(getHeight(root->left), getHeight(root->right));

    int balance = getBalance(root);

    if (balance > 1 && getBalance(root->left) >= 0)
        return rotateRight(root);

    if (balance > 1 && getBalance(root->left) < 0) {
        root->left = rotateLeft(root->left);
        return rotateRight(root);
    }

    if (balance < -1 && getBalance(root->right) <= 0)
        return rotateLeft(root);

    if (balance < -1 && getBalance(root->right) > 0) {
        root->right = rotateRight(root->right);
        return rotateLeft(root);
    }

    return root;
}

void BinaryTree::deleteValue(int value) {
    root = deleteNode(root, value);
}

void BinaryTree::update(int oldVal, int newVal) {
    deleteValue(oldVal);
    insert(newVal);
}

```

```

void BinaryTree::inorder(Node* node) {
    if (node == nullptr) return;
    inorder(node->left);
    cout << node->data << " ";
    inorder(node->right);
}

void BinaryTree::preorder(Node* node) {
    if (node == nullptr) return;
    cout << node->data << " ";
    preorder(node->left);
    preorder(node->right);
}

void BinaryTree::postorder(Node* node) {
    if (node == nullptr) return;
    postorder(node->left);
    postorder(node->right);
    cout << node->data << " ";
}

void BinaryTree::inorder() { inorder(root); cout << endl; }
void BinaryTree::preorder() { preorder(root); cout << endl; }
void BinaryTree::postorder() { postorder(root); cout << endl; }

```

### *main.cpp*

```

#include <iostream>
#include "tree.h"
#include "tree.cpp"

using namespace std;

int main() {
    BinaryTree tree;

    cout << "=== INSERT DATA ===" << endl;
    tree.insert(10);
    tree.insert(15);
    tree.insert(20);
    tree.insert(30);
    tree.insert(35);
    tree.insert(40);
    tree.insert(50);

    cout << "Data yang diinsert: 10, 15, 20, 30, 35, 40, 50 " << endl;

    cout << "\nTraversal setelah insert: " << endl;
    cout << "Inorder   : "; tree.inorder();
    cout << "Preorder  : "; tree.preorder();
    cout << "Postorder : "; tree.postorder();

    cout << "\n=== UPDATE DATA ===" << endl;
    cout << "Sebelum update (20 -> 25): " << endl;
    cout << "Inorder   : "; tree.inorder();

    tree.update(20, 25);

    cout << "Setelah update (20 -> 25): " << endl;
    cout << "Inorder   : "; tree.inorder();

    cout << "\n=== DELETE DATA ===" << endl;
    cout << "Sebelum delet (hapus subtree dengan root = 30): " << endl;
    cout << "Inorder   : "; tree.inorder();

    tree.deleteValue(30);

    cout << "Setelah delet (subtree root = 30 dihapus): " << endl;
    cout << "Inorde  : "; tree.inorder();

    return 0;
}

```

Screenshots Output

```

=== INSERT DATA ===
Data yang diinsert: 10, 15, 20, 30, 35, 40, 50

Traversal setelah insert:
Inorder   : 10 15 20 30 35 40 50
Preorder  : 30 15 10 20 40 35 50
Postorder : 10 20 15 35 50 40 30

=== UPDATE DATA ===
Sebelum update (20 -> 25):
Inorder   : 10 15 20 30 35 40 50
Setelah update (20 -> 25):
Inorder   : 10 15 25 30 35 40 50

=== DELETE DATA ===
Sebelum delet (hapus subtree dengan root = 30):
Inorder   : 10 15 25 30 35 40 50
Setelah delet (subtree root = 30 dihapus):
Inorde : 10 15 25 35 40 50

```

Deskripsi:

Program diatas mengimplementasikan struktur data Binary Search Tree (BST), pada struct node ini mempresentasikan node dalam pohon biner, kemudian pada Class BinaryTree ini mempresentasikan operasi pada pohon biner, kemudian pada fungsi main terdapat data 10 15 20 30 35 40 50, program melakukan update data pada angka 20 menjadi 25, kemudian meminta angka 30 dihapus dan akan menampilkan seluruh data yang tersedia saat ini.

- C. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)  
 Unguided 1

*bstree.h*

```

#ifndef BSTREE_H
#define BSTREE_H

#define Nil NULL

typedef int infotype;

typedef struct Node {
    infotype info;
    struct Node *left;
    struct Node *right;
} Node;

typedef Node *address;

address alokasi(infotype x);

void insertNode(address &root, infotype x);

address findNode(infotype x, address root);

void printInorder(address root);

int hitungJumlahNode(address root);

int hitungTotalInfo(address root);

int hitungKedalaman(address root, int start);

void printPreorder(address root);

void printPostorder(address root);
#endif

```

*bstree.cpp*



```

#include "bstree.h"
#include <iostream>

int max(int a, int b) {
    return (a > b) ? a : b;
}

address alokasi(infotype x) {
    address newNode = new Node;
    if (newNode != Nil) {
        newNode->info = x;
        newNode->left = Nil;
        newNode->right = Nil;
    }
    return newNode;
}

void insertNode(address &root, infotype x) {
    if (root == Nil) {
        root = alokasi(x);
    } else {
        if (x < root->info) {
            insertNode(root->left, x);
        } else if (x > root->info) {
            insertNode(root->right, x);
        }
    }
}

```

```

address findNode(infotype x, address root) {
    if (root == Nil) {
        return Nil;
    }

    if (x == root->info) {
        return root;
    } else if (x < root->info) {
        return findNode(x, root->left);
    } else {
        return findNode(x, root->right);
    }
}

void printInorder(address root) {
    if (root != Nil) {
        printInorder(root->left);
        std::cout << root->info << " - ";
        printInorder(root->right);
    }
}

int hitungJumlahNode(address root) {
    if (root == Nil) {
        return 0;
    }
    return 1 + hitungJumlahNode(root->left) + hitungJumlahNode(root->right);
}

int hitungTotalInfo(address root) {
    if (root == Nil) {
        return 0;
    }
    return root->info + hitungTotalInfo(root->left) + hitungTotalInfo(root->right);
}

```

```

int hitungKedalaman(address root, int currentDepth) {
    if (root == Nil) {
        return currentDepth - 1;
    }

    int leftDepth = hitungKedalaman(root->left, currentDepth + 1);
    int rightDepth = hitungKedalaman(root->right, currentDepth + 1);

    return max(leftDepth, rightDepth);
}

void printPreorder(address root) {
    if (root != Nil) {
        std::cout << root->info << " - ";
        printPreorder(root->left);
        printPreorder(root->right);
    }
}

void printPostorder(address root) {
    if (root != Nil) {
        printPostorder(root->left);
        printPostorder(root->right);
        std::cout << root->info << " - ";
    }
}

```

*main.cpp*

```

#include <iostream>
#include "bstree.h"
#include "bstree.cpp"

using namespace std;

int main() {
    cout << "Hello World" << endl;
    address root = Nil;

    insertNode(root, 1);
    insertNode(root, 2);
    insertNode(root, 6);
    insertNode(root, 4);
    insertNode(root, 5);
    insertNode(root, 3);
    insertNode(root, 6);
    insertNode(root, 7);

    printInorder(root);

    cout << "\nKedalaman: " << hitungKedalaman(root, 1) << endl;
    cout << "Jumlah Node: " << hitungJumlahNode(root) << endl;
    cout << "Total: " << hitungTotalInfo(root) << endl;

    cout << "PreOrder: ";
    printPreorder(root);

    cout << "\nPostOrder: ";
    printPostorder(root);

    return 0;
}

```

#### Screenshots Output

```
Hello World
1 - 2 - 3 - 4 - 5 - 6 - 7 -
Kedalaman: 5
Jumlah Node: 7
Total: 28
PreOrder: 1 - 2 - 6 - 4 - 3 - 5 - 7 -
PostOrder: 3 - 5 - 4 - 7 - 6 - 2 - 1 -
```

#### Deskripsi:

Program diatas mengimplementasikan struktur data Binary Search Tree (BST), pada bstree.cpp ini bertujuan untuk membuat node baru, menyisipkan nilai x, mencari node dengan nilai x, kemudian mengunjungi semua node secara berurutan, kemudian dihitung total nilainya dan jalur terpanjang pada tree, pada main.cpp memasukan angka 1,2,3,4,5,6,7,8 dan akan menghasilkan seperti di output atas.

#### D. Kesimpulan

Praktikum ini berhasil mengimplementasikan Binary Search Tree (BST) dasar dengan fungsi-fungsi operasional seperti alokasi node, penyisipan rekursif, pencarian, dan tiga mode traversal (Inorder, Preorder, Postorder).

#### E. Referensi

Long, Sean. "Binary Search Tree" (PPT). Data Structures and Algorithms Visualization- A PowerPoint Slides Based Approach. SUNY Oneonta.

Jarc, Duane J. (3 December 2005). "Binary Tree Traversals". Interactive Data Structure Visualizations. University of Maryland. Archived from the original on 27 February 2014. Retrieved 30 April 2006.